

Technical Requirements Document

Product: ExamGuard — Smart Exam Allocation & Identity Verification System **Document Version:** 1.0 **Date:** August 14, 2026 **Status:** Draft for Review

1. System Overview

ExamGuard is composed of three client surfaces sharing a common backend: an Admin Web Console and Student Web Portal (both Next.js), and an Invigilator Mobile Scanner App (Expo/React Native). All three talk to a Node.js backend exposing REST and WebSocket APIs, backed by PostgreSQL for relational data and Redis for token validation, session caching, and live dashboard pub/sub.



2. Technology Stack

Layer	Technology	Rationale
Invigilator Mobile App	React Native via Expo / Expo Go	Cross-platform iOS/Android from one codebase; expo-camera and expo-barcode-scanner give native camera access without ejecting; expo-sqlite provides offline storage
Student & Admin Web Apps	Next.js (React) + Tailwind CSS	Server-side rendering for fast first paint, responsive design, no app-store dependency for students
Backend API	Node.js with NestJS (or Go as an alternative)	NestJS gives modular, testable architecture with first-class WebSocket and guard/middleware support for auth and rate limiting
Database	PostgreSQL 15+	Strong relational integrity for the student-exam-hall-seat mapping model, native UUID and JSON support
Cache / Realtime	Redis 7+	Sub-millisecond token validation, pub/sub backbone for the live attendance dashboard
Object Storage	S3-compatible storage (AWS S3 / MinIO)	Stores and serves student photographs behind signed, expiring URLs
Auth	JWT (access + refresh tokens), bcrypt/argon2 password hashing	Stateless API auth; refresh tokens rotated and stored server-side for revocation
Realtime transport	WebSocket (Socket.IO or native ws)	Live dashboard updates and reveal-trigger push to student portal
CI/CD	GitHub Actions + Docker	Reproducible builds, containerized deployment across environments
Hosting	Containerized on a managed Kubernetes or PaaS (e.g., AWS ECS/EKS, Render, Railway)	Horizontal scalability for exam-day traffic spikes

3. Architecture Principles

- **Server-authoritative time.** All reveal-threshold and token-expiry decisions are computed and signed server-side. Client clocks are never trusted for security-relevant timing.
- **Offline-first invigilator client.** The Expo app treats the network as unreliable by default: it pre-syncs the full hall roster and photo set before the exam window, writes every scan to local SQLite first, and reconciles with the server opportunistically.
- **Single-use, signed tokens.** Every barcode encodes a JWT-like signed token scoped to one `student_exam_mapping` row. The server (or offline cache) rejects a token that has already been marked used.
- **Idempotent sync.** Offline-queued verification events carry a client-generated UUID so that repeated sync attempts never create duplicate audit records.
- **Least-privilege data exposure.** Hall and seat fields are withheld from all API responses to the student client until the reveal threshold has passed, not merely hidden in the UI — this prevents inspection-panel leakage.

4. API Design

4.1 Conventions

- REST, JSON payloads, versioned under `/api/v1/`.
- Auth via `Authorization: Bearer <JWT>` header; refresh via `/api/v1/auth/refresh`.
- Errors follow a consistent envelope: `{ "error": { "code": "STRING", "message": "STRING" } }`.
- Pagination via `?page=&pageSize=` with `X-Total-Count` response header.

4.2 Core Endpoints

Method	Path	Description	Consumer
POST	/api/v1/auth/login	Student or admin login	Student Portal, Admin Console
POST	/api/v1/auth/refresh	Rotate access token	All clients
POST	/api/v1/admin/students/bulk-upload	CSV/Excel roster ingestion	Admin Console
POST	/api/v1/admin/halls	Create exam hall	Admin Console
POST	/api/v1/admin/exams	Create exam	Admin Console
POST	/api/v1/admin/mappings/bulk	Bulk student-exam-hall-seat mapping	Admin Console
GET	/api/v1/admin/dashboard/:examId	Live attendance rollup	Admin Console
GET	/api/v1/student/exams	List mapped exams for logged-in student	Student Portal
GET	/api/v1/student/exams/:examId/pass	Dynamic barcode payload (hall/seat withheld pre-reveal)	Student Portal
GET	/api/v1/invigilator/halls/:hallId/roster	Pre-cache roster + photo URLs for offline use	Invigilator App
POST	/api/v1/invigilator/verify	Submit a scan verification (single or batched offline sync)	Invigilator App
POST	/api/v1/invigilator/verify/sync	Bulk sync of queued offline verification events	Invigilator App
GET	/api/v1/invigilator/students/search?roll=	Manual roll-number lookup	Invigilator App

4.3 WebSocket Channels

Channel	Purpose
<code>dashboard:{examId}</code>	Pushes live verification counts to the Admin Console
<code>reveal:{studentId}</code>	Pushes the unlock event to the Student Portal at T-30

5. Dynamic Barcode & Reveal Mechanism

1. On mapping creation, the backend generates a signed `barcode_token` (JWT with `mapping_id`, `exam_id`, `iat`, short `exp`) stored in `student_exam_mappings.barcode_token`.
2. The student portal requests a fresh, short-lived display token every 60–120 seconds via `/api/v1/student/exams/:examId/pass`; the server re-signs a rotating token bound to the same mapping so a static screenshot expires quickly.
3. At T-30 minutes (computed server-side against `exams.start_time`), a scheduled job (BullMQ/cron on Redis) flips a `reveal_unlocked` flag and emits a `reveal:{studentId}` WebSocket event; hall/seat fields are only included in API responses once this flag is true.
4. On first successful invigilator scan, the server marks the mapping `used_at = now()`; subsequent scans of the same token are rejected with a distinct `ALREADY_USED` error surfaced as a red flash in the app.

6. Offline Sync Design (Invigilator App)

- Pre-exam sync (triggered manually or 60 minutes before exam start while connected): downloads the full hall roster, including photo thumbnails, into `expo-sqlite` tables mirroring `student_exam_mappings` for that hall/exam.
- Each scan writes a local `verification_logs` row with `sync_status = false` and a client UUID.
- A background sync task, triggered on network-state change (via `@react-native-community/netinfo`) or app foregrounding, POSTs

- unsynced rows in batches to `/api/v1/invigilator/verify/sync` .
- Server applies rows idempotently keyed on the client UUID and returns per-row acceptance status; accepted rows are marked `sync_status = true` locally.
 - Conflict handling: if two invigilators offline-scan the same mapping (rare, e.g. device handoff), the server keeps the earliest `verified_at` and flags the later one `FLAGGED` for admin review.

7. Security Requirements

- Passwords hashed with argon2id; minimum complexity enforced at signup/reset.
- All barcode tokens signed with a rotating server-held secret (or asymmetric keypair for verification-only offline validation on the invigilator device).
- Hall/seat fields excluded server-side (not just UI-hidden) from any response prior to reveal.
- Photo URLs served as short-lived signed URLs from object storage, never public.
- Rate limiting on login and scan-verification endpoints to prevent brute-force and token-guessing.
- All traffic over TLS 1.2+; HSTS enabled on web properties.
- Audit log (`verification_logs`) is append-only at the application layer; no update/delete endpoint exists for it.

8. Performance & Scalability

- Target: 50,000 concurrent students able to load their pass within a 30-minute reveal window without degradation — handled via Redis-cached exam/hall data and CDN-served static assets.
- Database indexes on `student_exam_mappings(exam_id, hall_id)` , `verification_logs(mapping_id)` , and `students(roll_number)` to keep dashboard and lookup queries sub-50ms.
- Horizontal scaling of the API tier behind a load balancer; WebSocket layer backed by Redis adapter for multi-instance pub/sub.

9. Environments

Environment	Purpose
Local (Docker Compose)	Developer machines; Postgres, Redis, API, and both web apps containerized
Staging	Pre-production validation with anonymized data, used for UAT before each exam cycle
Production	Live institutional deployment, blue/green deploys to avoid downtime during active exam windows

10. Monitoring & Observability

- Structured logging (JSON) shipped to a centralized log store.
- Metrics: scan latency, sync success rate, reveal-job execution time, API error rate — exposed via Prometheus-compatible endpoint and visualized in Grafana.
- Alerting on: reveal-job failures, elevated `ALREADY_USED` / `WRONG_HALL` rates, sync backlog exceeding a threshold during an active exam window.